

BOMBIES

Rebuilding the game in Godot 4

A step-by-step guide to moving Bombies from the browser version into the Godot game engine, with GDScript examples for every system: the hero, zombies, weapons and mods, Washington Park, the controller, settings, and exporting to Android, iPhone and the web.

Contents

1. How the move works

2. Install Godot and create the project

3. Project layout

4. Controls: Input Map, gamepads and remapping

5. The on-screen controller

6. The hero: movement, camera, stances

7. Characters: models, animation, faces, hair

8. Weapons, mods and recoil

9. Zombie AI

10. Maps: Washington Park and the garage

11. Water reflections, weather and lighting

12. Explosions, fire and knockdowns

13. Driving

14. Menus, loadout and settings

15. Audio and surround sound

16. Loading screen

17. Alley Shot

18. Exporting: Android, iPhone, web

19. Suggested build order

1. How the move works

Bombies today is a single web page written in JavaScript with the three.js 3D library. Godot cannot open that file directly, so the move is a rebuild: you recreate each system in Godot using GDScript (Godot's own language, similar to Python). What carries over is everything you designed: the rules, the controls, the maps, the menus, the numbers for every gun and mod, and the look.

Carries over as-is	Rebuilt in Godot	Gets better in Godot
Game rules, waves, scoring, cheats, loadout choices, mod stats, button mapping, menu layout, the BombaRules and Bombies artwork	All code (JavaScript to GDScript), characters (proper rigged models instead of shapes), the world (built from scenes and models)	Real rigged and motion-captured characters, real ragdoll physics, built-in navigation, real reflections and lighting, native Android and iPhone apps, controller support built in

Tip: keep the browser version open while you build. It is your reference for how every feature should feel: walk speeds, camera distance, recoil, menu flow.

2. Install Godot and create the project

- Download **Godot 4.3 or newer** (standard version, not .NET) from **godotengine.org**. It is free and needs no installer on Windows or Mac; just unzip and run.
- Click **New Project**, name it **Bombies**, choose an empty folder.
- Renderer: pick **Mobile** if phones are your main target, or **Forward+** for the best desktop graphics. You can switch later in Project Settings.
- Open **Project > Project Settings** and set the values in the table below.

Setting	Value	Why
Display > Window > Size	1080 x 2340, Orientation: Portrait	Matches your phone layout: game on top, controller below
Display > Window > Stretch > Mode	canvas_items, Aspect: expand	Menus and controller scale cleanly on any screen
Rendering > Anti Aliasing > MSAA 3D	2x (Mobile) / 4x (Forward+)	Smooth edges
Physics > Common > Physics Ticks Per Second	60 (raise to 120 for 120 Hz phones)	Steady movement and shooting
Application > Config > Icon	Your zombie-bomb icon	App icon on every platform
Application > Boot Splash > Image	BombaRules logo	Shows instantly while the game starts

3. Project layout

Godot builds games from **scenes** (saved .tscn files made of nodes) and **scripts** (.gd files attached to nodes). This layout keeps Bombies organized:

```
res://
  autoload/        Game.gd  Settings.gd  Audio.gd  Loader.gd      (always-running managers)
  scenes/
    main/          Main.tscn  Title.tscn  Loading.tscn
```

```
  player/        Hero.tscn  Hero.gd  ViewModel.tscn
  zombies/       Zombie.tscn  Zombie.gd  ZombieSpawner.gd
  weapons/       Weapon.gd  rifle_m21.tres  shotgun.tres  magnum357.tres
  maps/          WashPark.tscn  Garage.tscn  water/Water.gdshader
  vehicles/      Car.tscn  Car.gd
  ui/            HUD.tscn  TouchPad.tscn  Menus.tscn  Loadout.tscn  Settings.tscn
  minigames/     AlleyShot.tscn
assets/          models/  animations/  textures/  audio/  fonts/BebasNeue.ttf
```

Autoloads (Project Settings > Globals > Autoload) are scripts that exist for the whole game. Add **Game.gd** (waves, score, state), **Settings.gd** (saving options), **Audio.gd** (buses, music) and **Loader.gd** (loading screen). Any script can then call `Settings.fov` or `Game.start_wave()`.

4. Controls: Input Map, gamepads and remapping

In Godot every control is an **action**. Open **Project Settings > Input Map** and add these actions, each with a keyboard key and a controller button. Godot reads Bluetooth and USB controllers (Xbox, PlayStation, 8BitDo and others) automatically once they are paired with the phone or computer.

Action	Keyboard	Controller	Bombies use
move_forward / back / left / right	W S A D	Left stick	Walk, sprint, sidestep
look_up / down / left / right	Mouse / arrow keys	Right stick	Aim the camera
shoot	Left mouse / Space	Right trigger, B	Fire
aim	Right mouse	Left trigger, D-pad left	Look down the sights
reload	R	A	Reload, clear jam
grenade	Q	X	Overhand throw
use	E	Y	Drive, get out, jump when nothing to use
jump / crouch	V / C	Y / D-pad down	Jump, crouch, slide, prone
light / shoulder	F / Tab	D-pad up / View	Flashlight, swap shoulder
next_weapon / pause	1-3 / Esc	D-pad right / Menu	Switch gun, pause

Reading the sticks with a dead zone that matches your **Drift 1-5** setting:

```
func _physics_process(delta):
    var dz := [0.06, 0.1, 0.15, 0.21, 0.28][Settings.drift - 1]
    var move := Input.get_vector("move_left", "move_right", "move_forward", "move_back", dz)
    var look := Input.get_vector("look_left", "look_right", "look_up", "look_down", dz)
```

Remapping (your Controller map screen): change an action's event at runtime and save it.

```
func remap(action: String, new_button: int) -> void:
    for ev in InputMap.action_get_events(action):
        if ev is InputEventJoypadButton:
            InputMap.action_erase_event(action, ev)
    var e := InputEventJoypadButton.new()
    e.button_index = new_button
    InputMap.action_add_event(action, e)
    Settings.cfg.set_value("bindings", action, new_button)
    Settings.save()
```

Controller list and pairing: `Input.get_connected_joypads()` returns connected controllers and `Input.get_joy_name(id)` gives names for your drop-down list. Connect to `Input.joy_connection_changed` to show "Controller connected". Pairing itself is always done in the

phone's or computer's Bluetooth settings.

5. The on-screen controller

Rebuild the white controller as a **CanvasLayer** scene (TouchPad.tscn) using your existing artwork exported as PNG images. Use **TouchScreenButton** nodes for A, B, X, Y, the D-pad, View and Menu, and set each one's **Action** property to the matching Input Map action, so touch, keyboard and controller all run the same code.

For the two sticks, use a small script on a Control node:

```
extends Control      # VirtualStick.gd
@export var action_left := "move_left"
@export var action_right := "move_right"
@export var action_up := "move_forward"
@export var action_down := "move_back"
var touch_id := -1
@onready var knob: Control = $Knob
func _gui_input(e):
    if e is InputEventScreenTouch:
        touch_id = e.index if e.pressed else -1
        if not e.pressed: _set(Vector2.ZERO)
    elif e is InputEventScreenDrag and e.index == touch_id:
        var v := (e.position - size / 2) / (size.x * 0.34)
        _set(v.limit_length(1.0))
func _set(v: Vector2):
    knob.position = size / 2 - knob.size / 2 + v * size.x * 0.34
    Input.action_press(action_right, max(v.x, 0)); Input.action_press(action_left, max(-v.x, 0))
    Input.action_press(action_down, max(v.y, 0));  Input.action_press(action_up, max(-v.y, 0))
```

Mirroring a Bluetooth controller on screen: each frame, read the real stick values with `Input.get_joy_axis()` and move the on-screen knobs to match; light up on-screen buttons while `Input.is_action_pressed()` is true.

Layout: for tall phones put the game in a full-screen SubViewportContainer and lay the controller over the bottom with its body at about 25% opacity (the "Over the game" layout). On desktop, scale the controller down to about 30% of the screen height.

6. The hero: movement, camera, stances

Make Hero.tscn with this node tree:

```
Hero (CharacterBody3D)              Hero.gd
  CollisionShape3D (Capsule 0.4 x 1.8)
  Model (your rigged character)     -> AnimationTree, Skeleton3D, hitbox Areas
  CameraPivot (Node3D at head height)
    SpringArm3D (length 3.2, shoulder offset 0.6)   third-person camera, stops at walls
      Camera3D
    FPSCamera (Camera3D at the eyes)               first-person
      ViewModel (arms + gun, own render layer)
```

The core movement script, with sprint, crouch, slide, prone, jump and knee landings:

```
extends CharacterBody3D
const WALK := 5.4; const SPRINT := 7.2; const CROUCH := 2.3; const PRONE := 0.9
var stance := "stand"; var slide_t := 0.0; var land_t := 0.0
@onready var pivot := $CameraPivot

func _physics_process(delta):
    var input := Input.get_vector("move_left", "move_right", "move_forward", "move_back")
    var aiming := Input.is_action_pressed("aim")
    var sprinting := input.y < -0.9 and stance == "stand" and not aiming
```

```
    var stand_speed := SPRINT if sprinting else WALK
    var speed: float = {"stand": stand_speed, "crouch": CROUCH, "prone": PRONE}[stance]
    var dir := (transform.basis * Vector3(input.x, 0, input.y)).normalized()
    if slide_t > 0:
        slide_t -= delta; speed = 9.0 * slide_t / 0.75
    velocity.x = dir.x * speed; velocity.z = dir.z * speed
    if not is_on_floor():
        velocity.y -= 22.0 * delta
    elif Input.is_action_just_pressed("jump") and stance == "stand":
        velocity.y = 5.8
    var falling := velocity.y
    move_and_slide()
    if is_on_floor() and falling < -8.5:
        land_t = 0.38          # drop to a knee
    if Input.is_action_just_pressed("crouch"):
        if sprinting: slide_t = 0.75; stance = "crouch"
        elif Input.is_action_pressed("shoot"): stance = "prone"   # drop shot
        else: stance = "crouch" if stance == "stand" else "stand"
```

Look and recoil: rotate the hero left and right, the CameraPivot up and down. Add the gun's recoil to a pending value and bleed it into the pitch, then ease most of it back when firing stops:

```
var recoil_pending := 0.0; var recoil_total := 0.0
func add_recoil(amount: float):   # called by the weapon on every shot
    recoil_pending += amount
func _process(delta):
    var look := Input.get_vector("look_left", "look_right", "look_up", "look_down")
    var sens := Settings.sens_x * (0.55 if aiming else 1.0)
    rotate_y(-look.x * 2.4 * sens * delta)
    var climb := recoil_pending * min(1.0, delta * 22)
    recoil_pending -= climb; recoil_total += climb
    if not Input.is_action_pressed("shoot"):
        var back := recoil_total * min(1.0, delta * 6)
        recoil_total -= back; climb -= back * 0.8
    pivot.rotation.x = clamp(pivot.rotation.x - look.y * 1.5 * delta + climb, -0.85, 0.6)
```

Field of view: set `Camera3D.fov` from the slider (80-120). Godot's fov is vertical by default; set **Keep Aspect** to **Keep Width** so the slider means horizontal degrees like in Bombies. **First/third person:** switch `current` between the two cameras and hide the body model in first person.

7. Characters: models, animation, faces, hair

This is where Godot pays off. Instead of shapes built in code, use real rigged 3D characters.

Getting the models

- **Your hero:** make a head from your photos with a photo-to-3D tool (such as KeenTools FaceBuilder for Blender), add the suit, beard, sunglasses and slicked-back hair in Blender, then rig it with Mixamo (free) or Rigify.
- **Zombies:** start from free or paid rigged human models, then paint rot, grime and wounds on the textures in Blender or Substance Painter. Keep 4-6 variants with different hair and clothes.
- **Animations:** Mixamo has free walk, run, crouch, prone, slide, jump, landing, rifle and pistol aim poses, zombie walks, feeding, attacks, falls and get-ups.
- Export from Blender as **.glb** and drag the file into Godot's FileSystem panel.

Animation

Add an **AnimationTree** with a state machine: Locomotion (a BlendSpace2D of idle, walk, run and strafe), Crouch, Prone, Slide, Jump, Land, Knockdown, GetUp, Throw and Drive. Drive it from Hero.gd:

```
@onready var anim: AnimationTree = $Model/AnimationTree
func update_animation():
    var local_v := global_transform.basis.inverse() * velocity
    anim.set("parameters/Locomotion/blend_position", Vector2(local_v.x, -local_v.z) / SPRINT)
    anim["parameters/conditions/crouching"] = stance == "crouch"
    anim["parameters/conditions/prone"] = stance == "prone"
```

Hands on guns: use **SkeletonIK3D** (or the newer IK modifiers) so the support hand always grips the forend marker on each gun: cupped under the M21, on the pump of the shotgun, and wrapped over the shooting hand for the Magnum's thumbs-back grip.

Faces

If your head model has the 52 ARKit blendshapes (FaceBuilder and many tools can add them), animate them by name, exactly like the jaw, smile, snarl and squint system in the browser version:

```
@onready var face: MeshInstance3D = $Model/Skeleton3D/Head
func set_expression(smile: float, snarl: float, jaw: float):
    face.set_blend_shape_value(face.find_blend_shape_by_name("mouthSmileLeft"), smile)
    face.set_blend_shape_value(face.find_blend_shape_by_name("mouthSmileRight"), smile)
    face.set_blend_shape_value(face.find_blend_shape_by_name("noseSneerLeft"), snarl)
    face.set_blend_shape_value(face.find_blend_shape_by_name("jawOpen"), jaw)
```

Hair: make hair in Blender as curves or hair cards attached to the head bone, so it never floats away from the scalp.

8. Weapons, mods and recoil

Store each gun and mod as a **Resource**, so you edit numbers in the Inspector instead of in code.

```
class_name WeaponData extends Resource
@export var name := "M21"
@export var mag := 20; @export var reserve := -1   # -1 = infinite
@export var damage := 1.3; @export var pellets := 1
@export var fire_delay := 0.105; @export var spread := 0.006
@export var recoil := 0.016; @export var reload_time := 1.7
@export var model: PackedScene
@export var mods: Array[ModData] = []

class_name ModData extends Resource
@export var slot := "barrel"          # optic, barrel, magazine, ammo, grip, cylinder, finish
@export var label := "Extended barrel"
@export var damage_mult := 1.2; @export var spread_mult := 0.6; @export var recoil_mult := 1.0
@export var zoom := 1.0               # 1.5, 3, 5 for scopes
@export var part_scene: PackedScene   # the mesh that appears on the gun
```

Gun	Mods to recreate
M21	Optic: iron, 1.5x, 3x, 5x. Barrel: short, standard, extended, light. Magazine: 10 or 20
Pump shotgun	Ammo: buckshot, slugs, incendiary. Grip: pump, vertical (recoil x0.65), stub. Stock: synthetic or walnut
.357 Magnum	Barrel: snub, 6", 8.4". Cylinder: six or eight shot. Grip: rubber (recoil x0.8) or walnut. Finish: stainless or blued

Shooting: fire a ray from the camera center with `PhysicsRayQueryParameters3D`. Put Area3D hitboxes on the zombie's head, torso, arms and legs (attached with BoneAttachment3D), so headshots do triple

damage, two leg hits make a zombie crawl, and two arm hits leave that arm hanging.

```
func fire():
    var cam := get_viewport().get_camera_3d()
    var from := cam.global_position
    var stance_k := {"prone": 0.5, "crouch": 0.72}.get(hero.stance, 1.0)
    var spread := data.spread * stance_k
    var jitter := Vector3(randf_range(-spread, spread), randf_range(-spread, spread), 0)
    var dir := (-cam.global_basis.z + jitter).normalized()
    var q := PhysicsRayQueryParameters3D.create(from, from + dir * 200.0)
    var hit := get_world_3d().direct_space_state.intersect_ray(q)
    if hit and hit.collider.has_method("take_hit"):
        hit.collider.take_hit(data.damage, hit.collider.part)   # "head", "leg_l", ...
    hero.add_recoil(data.recoil * recoil_mult)
```

Scopes: when aiming with a 3x or 5x optic, divide the camera fov by the zoom and show a TextureRect scope overlay. **Mods glow in the loadout:** give the highlighted part a second material with an emission pulse.

9. Zombie AI

Godot's navigation replaces the hand-written path-finding grid. Add a **NavigationRegion3D** to each map and click **Bake NavMesh**; lakes and buildings are excluded automatically. Each zombie gets a **NavigationAgent3D** with **Avoidance** turned on, which spreads crowds out instead of lining them up.

```
extends CharacterBody3D      # Zombie.gd
enum {WANDER, INVESTIGATE, SEARCH, CHASE, ATTACK, FEED}
var state := WANDER; var hp := 4.0; var crawling := false
@onready var agent: NavigationAgent3D = $NavigationAgent3D
@onready var think := $ThinkTimer       # wait_time 0.2, autostart: decide 5x a second

func _on_think_timer_timeout():
    var hero := Game.hero
    var to_hero := hero.global_position - global_position
    var sees := to_hero.length() < 16 and (-global_basis.z).angle_to(to_hero) < deg_to_rad(55) \
        and _clear_line(hero) and not Cheats.ghost
    match state:
        WANDER, FEED, SEARCH, INVESTIGATE:
            if sees: state = CHASE
        CHASE:
            agent.target_position = hero.global_position + _crowd_slot_offset()
            if to_hero.length() < 1.2: state = ATTACK

func _physics_process(delta):
    var next := agent.get_next_path_position()
    var v := (next - global_position).normalized() * (0.8 if crawling else 1.4)
    agent.velocity = v          # avoidance computes the final velocity
func _on_velocity_computed(safe_v: Vector3):
    velocity = safe_v; move_and_slide()
```

- **Hearing:** when the hero fires, call a group method such as `get_tree().call_group("zombies", "hear", position, 28.0)`; zombies inside the radius switch to INVESTIGATE.
- **Feeding:** place bodies as Marker3D nodes in a "food" group; WANDER zombies within 14 m walk to one and play the kneel-and-eat animation, with reduced sight.
- **Sneak attacks:** double damage when the zombie is not in CHASE.
- **Performance:** the ThinkTimer already limits decisions; also use VisibilityNotifier3D to pause animation on far-away zombies.

10. Maps: Washington Park and the garage

Start each map as a quick blockout with **CSG** shapes (boxes and cylinders you can place right in the editor) to test layout and gameplay, then replace them with models from Blender.

Washington Park checklist

- Big lake with a natural shoreline, and a smaller southern lake; reeds and cattails at the edges (use MultiMeshInstance3D to scatter thousands cheaply).
- The boathouse: cream columns, white railings, brown hip roof with chimney, wide stairs, the railed walkway over the water, warm string lights (an emissive material plus a few OmniLight3D nodes).
- Fishing pier, gravel loop path, asphalt road with center line, formal flower garden, benches, black lamp posts with globe lights.
- Cottonwoods and maples in summer green and fall gold, houses around the edge, the brick high-rise and the Rockies on the horizon.
- Canada geese that float, graze and scatter.

Use a photo-scanned or hand-modeled boathouse for realism, and Godot's **Terrain3D** add-on (free, in the Asset Library) for grass and paths.

Map selection

Keep each map as its own scene and switch with `get_tree().change_scene_to_file("res://scenes/maps/WashPark.tscn")` behind the loading screen.

11. Water reflections, weather and lighting

Add a **WorldEnvironment** node to each map. Turn on **SSR** (screen-space reflections) for the lake and wet streets, **SSAO** for contact shadows, **Glow**, fog and a **PhysicalSkyMaterial** or a sky texture. On desktop (Forward+) you can also turn on **SDFGI** for realistic bounced light, the closest Godot gets to the ray-traced look in the Unreal videos.

A simple lake shader with ripples and a mirror that is stronger at a glance (fresnel):

```
shader_type spatial;
render_mode cull_disabled;
uniform vec3 deep : source_color = vec3(0.12, 0.23, 0.24);
uniform sampler2D screen_tex : hint_screen_texture, filter_linear_mipmap;
void fragment() {
    vec2 ripple = vec2(sin(UV.y * 80.0 + TIME * 1.7), cos(UV.x * 70.0 + TIME * 1.3)) * 0.004;
    float fres = 0.28 + 0.62 * pow(1.0 - dot(NORMAL, VIEW), 3.0);
    ALBEDO = deep;
    ROUGHNESS = 0.05;
    METALLIC = 0.2;
    SPECULAR = fres;      // SSR supplies the upside-down reflection of trees and buildings
    NORMAL_MAP = vec3(0.5 + ripple.x * 30.0, 0.5 + ripple.y * 30.0, 1.0);
}
```

For a perfect mirror on phones, use a planar reflection: a second Camera3D mirrored under the water in a SubViewport, with its image fed into the shader. The Godot Asset Library has ready-made planar reflection add-ons.

- **Rain:** a GPUParticles3D box that follows the camera, plus wet materials (lower roughness) on the ground.

- **Lightning:** flash the DirectionalLight3D energy for a few frames, then play thunder after a short delay.
- **Failing garage power:** animate the OmniLight3D energies down wave by wave; the flashlight is a SpotLight3D on the gun.

12. Explosions, fire and knockdowns

- **Explosions:** one scene with GPUParticles3D for the fireball, sparks, smoke and debris, an OmniLight3D flash, a Decal scorch mark and camera shake. Grenades, barrels, cars and molotovs all spawn it.
- **Fire:** a flame shader on billboard quads, or a fire particle preset from the Asset Library. Burning characters swap to a charred material by raising a shader parameter.
- **Blood and bullet holes:** Decal nodes placed at the hit point and normal.

Thrown by a blast, then getting up: Godot has real ragdolls. In the Skeleton3D, click **Create Physical Skeleton** once to add PhysicalBone3D nodes, then:

```
func knockdown(from: Vector3):
    $Model/Skeleton3D/PhysicalBoneSimulator3D.physical_bones_start_simulation()
    var push := (global_position - from).normalized() * 9.0 + Vector3.UP * 6.0
    $Model/Skeleton3D/PhysicalBoneSimulator3D/Spine.apply_central_impulse(push)
    await get_tree().create_timer(1.6).timeout
    $Model/Skeleton3D/PhysicalBoneSimulator3D.physical_bones_stop_simulation()
    anim["parameters/playback"].travel("GetUp")     # kneel, gun up, stand
```

13. Driving

Use **VehicleBody3D** with four VehicleWheel3D nodes and a car model with see-through glass. Add a **Marker3D** at the driver's seat and one at the door. To get in, play an enter animation (walk to the door, right leg in, sit, hands on the wheel), reparent the hero to the seat marker, and switch control to the car:

```
func _physics_process(delta):
    if not occupied: return
    var input := Input.get_vector("move_left", "move_right", "move_forward", "move_back")
    engine_force = -input.y * 1600.0
    brake = 40.0 if input.y > 0.1 and linear_velocity.dot(-global_basis.z) > 1.0 else 0.0
    steering = move_toward(steering, -input.x * 0.5, delta * 2.5)
    $SteeringWheel.rotation.z = -steering * 3.0
```

Running zombies over: detect them with an Area3D on the bumper and trigger their ragdoll with an impulse in the car's direction.

14. Menus, loadout and settings

Build menus with **Control** nodes (VBoxContainer, Button, HSlider, CheckButton, OptionButton). Make one **Theme** resource with the Bebas Neue font, dark translucent panels, gold highlights and the red selection bar, and apply it to the root of every menu. Set **Focus Neighbors** so the controller and keyboard can move through every menu.

Screen	Contents
Title	Survive the Bombies, Map, Alley Shot, Settings
Loadout	Primary, secondary, lethal, perk rows; Mods button opens the attachment screen with the camera zoomed on the gun

Screen	Contents
Settings > Audio	Master, Music, Sound effects, Voice lines, Surround sound
Settings > Graphics	Quality (Smooth / High / Ultra), Field of view 80-120, View (first / third), Weather, Screen Hz, FPS limit
Settings > Controls	Drift, Invert look, Look sensitivity X and Y, Controller layout, Bluetooth controller on/off, Controller list, Controller map
Settings > Cheats	God mode, Infinite ammo, One-shot kills, Incendiary, Explosive, Big heads, Super speed, Slow motion (zombies only), No zombies, Ghost

Save everything with **ConfigFile** in Settings.gd:

```
extends Node       # Settings.gd (autoload)
var cfg := ConfigFile.new()
const PATH := "user://settings.cfg"
var fov := 80; var quality := "high"; var fps := 120; var drift := 3; var sens_x := 3
func _ready(): cfg.load(PATH); fov = cfg.get_value("graphics", "fov", 80); apply()
func save(): cfg.set_value("graphics", "fov", fov); cfg.save(PATH)
func apply():
    Engine.max_fps = fps
    get_viewport().scaling_3d_scale = {"smooth": 0.7, "high": 1.0, "ultra": 1.0}[quality]
    var shadow := {"smooth": 1024, "high": 2048, "ultra": 4096}[quality]
    RenderingServer.directional_shadow_atlas_set_size(shadow, true)
    get_viewport().msaa_3d = Viewport.MSAA_4X if quality == "ultra" else Viewport.MSAA_2X
```

Screen Hz: Godot renders at the display's rate when V-Sync is on; on Android, the export option and `DisplayServer.screen_get_refresh_rate()` let you read and match 90 or 120 Hz screens.

15. Audio and surround sound

- Create buses in the **Audio** tab: **Master**, **Music**, **SFX**, **Voice**. The volume sliders call `AudioServer.set_bus_volume_db(index, linear_to_db(value))`.
- **Surround:** use **AudioStreamPlayer3D** for every sound that happens in the world (groans, gunshots, explosions, car alarms, geese). Godot positions them around the listener automatically, and outputs true 5.1 or 7.1 when Project Settings > Audio > Driver > Output channels is set and the device supports it. With headphones, sounds still come from left, right, front and behind.
- The Camera3D acts as the listener; add an AudioListener3D to the hero's head if you want the hero's ears instead.
- Music: an AudioStreamPlayer on the Music bus, with a second layered track (the heartbeat) faded in while zombies are chasing.

16. Loading screen

Loading.tscn: a black background, your **BombaRules** logo in a TextureRect, a gold **ProgressBar**, and the text "Loading Bombies…". Load the map in the background and fill the bar with real progress:

```
extends Control     # Loading.gd
var path := "res://scenes/maps/WashPark.tscn"
func _ready(): ResourceLoader.load_threaded_request(path)
func _process(_d):
    var progress := []
    var status := ResourceLoader.load_threaded_get_status(path, progress)
    $ProgressBar.value = progress[0] * 100.0
    if status == ResourceLoader.THREAD_LOAD_LOADED:
```

```
get_tree().change_scene_to_packed(ResourceLoader.load_threaded_get(path))
```

17. Alley Shot

Make it a separate scene: a row of building facades on a Path3D that slides right to left with an ease-in-out tween and stops at each building. Characters are real animated models placed behind window openings (cut real holes in the facade mesh so they are actually inside). Tap or press B to shoot; touch input becomes a ray from the camera through the tap point with `camera.project_ray_origin()` and `project_ray_normal()`.

18. Exporting: Android, iPhone, web

Android (APK for your phone)

- **Editor > Manage Export Templates > Download**.
- Install Android Studio once (for the SDK) and OpenJDK 17, then point to them in **Editor Settings > Export > Android**. Godot can create the debug keystore for you.
- **Project > Export > Add > Android**: set package name (for example com.bombarules.bombies), app name Bombies, icons, portrait orientation, and Immersive Mode on. Click **Export Project** to get an .apk you can install, or an .aab for the Google Play Store.
- Use **Remote Debug** (the Android icon at the top right of the editor) to run the game straight on your phone over USB while you build.

iPhone and iPad

- Needs a Mac with Xcode and an Apple Developer account (free for testing on your own device, paid to publish).
- Export the iOS project from Godot, open it in Xcode, select your iPhone and press Run. TestFlight lets friends try it.

Web (keep using GitHub Pages)

- Export with the **Web** preset. Turn **Thread Support** off, so it runs on GitHub Pages without special server headers.
- Upload the exported files (index.html, .wasm, .pck, .js) to your bombies repository. Note that Godot web builds are much larger than the current page and run best on desktop and newer phones.

19. Suggested build order

Build in this order, so you always have something playable:

#	Milestone	Done when
1	Project, Input Map, a blockout Washington Park	A capsule can walk around the park with keyboard, controller and touch
2	Hero with camera, stances and the M21	You can sprint, crouch, slide, jump and shoot targets with recoil

#	Milestone	Done when
3	One zombie type with navigation and the behavior states	Zombies wander, hear shots, chase, surround and attack
4	Waves, HUD, health, pickups, title and pause menus	A full round of play from title screen to game over
5	Real character models and animations	Your hero and zombies replace the capsules
6	All three guns, mods and the loadout	Mods change stats and appear on the gun
7	Map art, water, weather, lighting	The park looks like your photos
8	Explosions, fire, ragdolls, grenades, driving	Big moments feel right
9	Settings, audio, loading screen, controller map	Every option from the browser version works
10	Alley Shot, garage map, polish, export	Installed on your phone and on GitHub Pages

Where to learn more: the official Godot documentation at docs.godotengine.org (start with “Your first 3D game”), the Godot Asset Library inside the editor for free add-ons, and Mixamo for free animations. Ask me any time for the full GDScript for a specific system and I will write it out.